JAVA INTERVIEW PREPARATION

CHAPTER 77

BEHAVIORAL, DOMAIN, AND RESILIENCE PATTERNS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Behavioral, Domain, and Resilience Patterns

Senior / Lead Java Developer Reading Chapter

Behavioral patterns organize decisions, workflows, state transitions, and communication between objects. Domain patterns protect the language and transaction boundaries of the business. Resilience patterns control failure across time and distributed dependencies. In production Java systems these groups meet constantly: a Strategy selects policy, a Command records intent, a State machine governs transitions, a Repository loads an aggregate, and a retry or circuit breaker surrounds an external call.

The senior-level skill is not memorizing diagrams. It is identifying the force that varies, the invariant that must remain true, and the failure modes introduced by the abstraction. Patterns are design vocabulary, not proof that the design is correct.

A map from pressure to pattern

```
behavior varies by policy             -> Strategy
workflow skeleton is stable           -> Template Method
request must become an explicit value -> Command
handlers form an ordered pipeline      -> Chain of Responsibility
behavior changes with lifecycle state -> State
many listeners react to an event       -> Observer
colleagues need coordinated routing    -> Mediator
rules must compose                     -> Specification
domain objects need persistence access -> Repository + Unit of Work
temporary failure may recover          -> Retry
repeated failure must stop calls       -> Circuit Breaker
one dependency must not consume all capacity -> Bulkhead
database change and event must agree   -> Transactional Outbox
```

The same problem can involve several rows. Keep each responsibility explicit so a retry mechanism does not also become a business state machine and a repository does not also become an integration facade.

Strategy: make a policy replaceable

Strategy encapsulates interchangeable algorithms behind a stable contract. It fits pricing rules, routing, validation policy, allocation, serialization, and provider selection.

```
@FunctionalInterface
interface DiscountPolicy {
    Money discount(Order order);
}

DiscountPolicy loyalty = order ->
        order.customer().isLoyal() ? order.subtotal().percent(5) : Money.ZERO;

final class Checkout {
    private final DiscountPolicy discountPolicy;

    Total calculate(Order order) {
        return new Total(order.subtotal().minus(discountPolicy.discount(order)));
    }
}
```

Lambdas make small stateless strategies inexpensive. A named class remains useful when the algorithm has configuration, dependencies, metrics, or nontrivial tests. Avoid selecting a strategy through scattered `if` statements; centralize selection in configuration, a registry, or a factory.

A strategy contract needs more than a method signature. Define whether it is thread-safe, deterministic, side-effect-free, allowed to decline, and expected to be fast. If a pricing strategy calls a remote service, its latency and failure semantics belong in the policy contract. A registry keyed by user-controlled text must reject unknown or unapproved implementations rather than reflectively loading arbitrary classes.

Strategy supports the Open/Closed Principle only when the abstraction corresponds to a real variation. Creating five interfaces around a rule that never changes adds indirection without protecting anything.

Template Method: preserve a workflow skeleton

Template Method defines the stable sequence in a base type and lets subclasses override selected steps.

```
abstract class ImportJob {
    public final ImportResult run(InputStream input) {
        RawData raw = read(input);
        ValidatedData valid = validate(raw);
        ImportResult result = persist(valid);
        publish(result);
        return result;
    }

    protected abstract ValidatedData validate(RawData raw);
    protected abstract ImportResult persist(ValidatedData data);
    protected void publish(ImportResult result) { }
}
```

The final template preserves ordering. Hooks provide controlled variation. The cost is inheritance coupling: subclasses depend on protected details and may violate assumptions the base class did not express.

Strategy uses composition and can vary an algorithm at runtime. Template Method uses inheritance to vary steps inside a stable workflow. Modern Java often replaces a deep template hierarchy with a final orchestrator that accepts strategies for validation, persistence, or notification. Template Method remains reasonable when subclasses share a strong lifecycle and the allowed extension points are deliberately narrow.

Command: represent an intention as data

A Command packages a request and the information required to execute it. Commands are useful for queues, retries, audit, scheduling, authorization, undo, and decoupling an invoker from a handler.

```
record TransferFunds(
        UUID commandId,
        AccountId from,
        AccountId to,
        Money amount,
        Instant requestedAt) { }

interface CommandHandler<C> {
    void handle(C command);
}
```

Once a command crosses a process boundary, it becomes a versioned message contract. Include stable identity for deduplication, validate authorization in the execution context, and avoid serializing arbitrary Java object graphs. Do not assume that placing a command on a queue makes its side effect exactly once. Consumers can receive duplicates after a crash, so handlers should be idempotent or record processed command identifiers with the state change.

Undo is not simply calling the opposite method. Real operations may involve fees, notifications, external settlement, or intervening changes. Model compensation as another explicit command whose business rules are understood.

Chain of Responsibility: an ordered processing pipeline

Chain of Responsibility passes a request through handlers until one handles it or the chain finishes. Servlet filters, security checks, validation pipelines, and middleware are familiar forms.

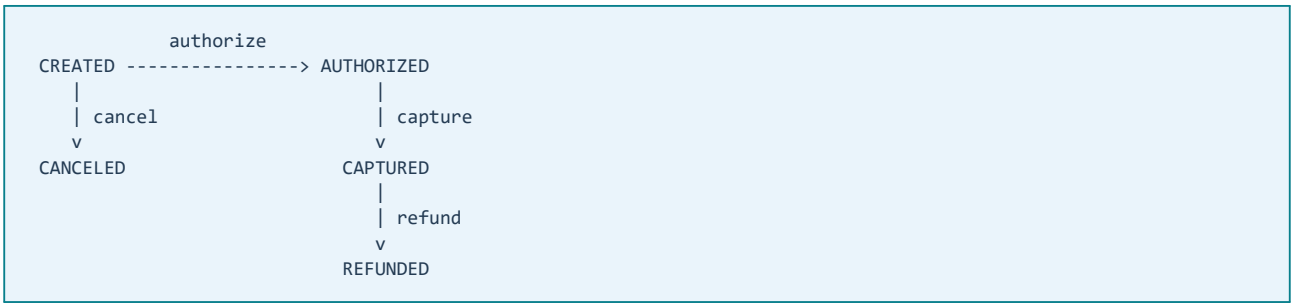
```
interface OrderCheck {
    CheckResult check(Order order);
}

final class ValidationPipeline {
    private final List<OrderCheck> checks;

    CheckResult validate(Order order) {
        for (OrderCheck check : checks) {
            CheckResult result = check.check(order);
            if (result.rejected()) return result;
        }
        return CheckResult.accepted();
    }
}
```

Define continuation semantics clearly. Does every handler run, does the first match stop, or are errors accumulated? Is ordering configuration or code? Can a handler mutate the request? Can an asynchronous handler resume on another thread? Ambiguity causes security checks to be skipped or validation errors to appear inconsistently.

Retries, transactions, and metrics can also form chains, but wrapper order changes behavior. Treat order as tested configuration and expose it in diagnostics.

State: put lifecycle-dependent behavior beside transitions

The State pattern represents behavior that changes with an object's lifecycle state. It replaces sprawling conditionals when each state has distinct allowed operations and transitions.



A sealed hierarchy makes legal states visible:

```
sealed interface PaymentState permits Created, Authorized, Captured, Refunded {
    PaymentState capture();
}

record Authorized(String authorizationId) implements PaymentState {
    public PaymentState capture() {
        return new Captured(authorizationId, Instant.now());
    }
}
```

Not every enum switch needs the State pattern. A small stable state machine may be clearer as an enum plus an explicit transition table. Use state objects when behavior is substantial, varies independently, or conditionals are scattered across methods.

Persistence introduces concurrency. Two workers may load `AUTHORIZED` and both try to capture. Optimistic locking, a unique command key, or database locking must enforce the transition atomically. The in-memory

pattern alone does not guarantee workflow correctness. Persist both state and transition evidence, and define how unknown future states are handled by older consumers.

Observer and domain events

Observer lets subscribers react when a subject publishes an event. In-process application events can decouple secondary reactions such as metrics or cache invalidation. Distributed domain events communicate facts across service boundaries.

```
record OrderPlaced(OrderId orderId, CustomerId customerId, Instant occurredAt) { }

interface DomainEventHandler<E> {
    void on(E event);
}
```

Synchronous observers run in the publisher's call path. They are simple and can share the transaction, but a slow or failing listener affects the original operation. Asynchronous observers isolate latency but introduce eventual consistency, ordering, duplicate delivery, queue capacity, and trace propagation.

Publish facts in past tense. `OrderPlaced` says something happened; `SendEmail` is a command requesting an action. This distinction helps ownership: many observers may react to a fact, while one handler normally owns a command.

In-process observer registration can leak memory if long-lived publishers retain short-lived listeners. Provide unsubscription or lifecycle-managed registration. In distributed systems, use durable brokers when delivery matters, define partition keys for ordering, and make consumers idempotent. Backpressure is part of the design; an unbounded executor is not a reliable event bus.

Mediator: centralize collaboration without centralizing all logic

Mediator coordinates communication among components that would otherwise reference one another densely. A request dispatcher, workflow coordinator, or UI controller can act as a mediator.

The benefit is reduced coupling between colleagues. The risk is a god mediator containing every business rule and knowing every component. Keep routing and orchestration in the mediator while decisions remain in domain objects or strategies. Make the interaction traceable: when all calls pass through one coordinator, it is an excellent place for correlation, but not a reason to hide causal relationships.

Iterator: control traversal independently of representation

Iterator exposes sequential access without revealing the collection's internal structure. Java's `Iterable`, enhanced `for`, collections, spliterators, and streams build on this idea.

The production questions concern consistency. A fail-fast collection iterator detects some concurrent structural changes but is a debugging aid, not a concurrency guarantee. A snapshot iterator gives stable traversal at memory cost. A weakly consistent concurrent iterator may reflect some changes without throwing but does not represent one atomic instant.

Do not return an iterator over a resource-backed result set without defining closure. A stream backed by database rows or files should usually be used in try-with-resources. Also distinguish internal iteration, where a stream controls traversal, from external iteration, where the caller explicitly advances and can pause, interleave, or stop.

Specification: compose business predicates

Specification represents a business rule that can be combined with other rules.

```
interface Specification<T> {
    boolean isSatisfiedBy(T candidate);

    default Specification<T> and(Specification<T> other) {
        return value -> isSatisfiedBy(value) && other.isSatisfiedBy(value);
    }
}
```

Specifications help when rules need names, reuse, composition, and independent tests. `EligibleForCredit` communicates intent better than a repeated boolean expression. They can also explain rejection by returning structured results rather than a bare boolean.

A common trap is assuming an arbitrary Java predicate can be translated into SQL. In-memory execution and database query generation are different interpreters. A query specification must use an expression model understood by JPA Criteria, QueryDSL, or another translator. Keep domain meaning stable while allowing separate evaluation mechanisms, and verify that database and in-memory semantics agree for nulls, time zones, and collation.

Repository and Unit of Work

A Repository provides collection-like access to aggregates in domain terms. It hides persistence mechanics without pretending that every query is free.

```
interface OrderRepository {
    Optional<Order> find(OrderId id);
    List<Order> awaitingDispatch(Instant before, int limit);
    void save(Order order);
}
```

This interface expresses domain intent. A universal `GenericRepository<T, ID>` often leaks persistence operations, encourages loading whole aggregates for reporting, and cannot express each aggregate's meaningful queries.

The Unit of Work tracks changes and commits them as one transaction. JPA's persistence context behaves like a Unit of Work and identity map: within a context, the same entity identity normally maps to one managed instance, changes are detected, and SQL may be delayed until flush or commit.

```
transaction begins
      |
repository loads aggregates
      |
domain changes managed objects
      |
flush: dirty checking generates SQL
      |
commit or rollback
```

Repository boundaries should align with aggregate consistency boundaries. Cross-aggregate reports may use dedicated queries or projections instead of forcing everything through aggregate repositories. Keep transactions at application use-case boundaries, control lazy loading, and avoid exposing managed entities directly through REST where serialization can trigger accidental queries.

Retry: repeat only when another attempt can succeed safely

Retry handles transient failure such as a brief connection reset, lock conflict, or rate-limit response. It is not a response to validation errors, authorization failures, or deterministic bugs.

```
attempt 1 fails transiently
      |
backoff + jitter
      v
attempt 2 succeeds
```

Safe retry requires idempotency or deduplication. A timed-out payment request may have succeeded remotely even though the response was lost. Repeating it without an idempotency key can charge twice. Bound retries by both attempt count and end-to-end deadline. Exponential backoff reduces pressure; jitter prevents many clients from retrying in lockstep.

Place retry at the layer that can classify failure and understand operation semantics. Avoid nested retry amplification. Three layers with three attempts each can produce twenty-seven physical calls. Record logical operations separately from attempts and expose exhaustion as a meaningful failure.

Circuit Breaker: stop spending capacity on repeated failure

A circuit breaker watches recent outcomes and temporarily rejects calls when a dependency appears unhealthy.

```
            failures exceed threshold
 CLOSED ------------------------------> OPEN
   ^                                     |
   | successful trial                    | wait duration
   |                                     v
   +------------------------------ HALF_OPEN
                               limited trial calls
```

Closed calls the dependency normally. Open fails fast, protecting threads and the dependency. Half-open permits a limited probe to decide whether recovery is real. Configure failure classification, sliding window, minimum call volume, open duration, and probe concurrency. A tiny sample should not trip a critical circuit, and business rejections should not count as dependency failure.

A breaker is not a health oracle. It represents one caller's recent experience and can differ by instance. Fallbacks must be honest: stale catalog data may be acceptable, but inventing a successful payment is not. Monitor state changes and rejected calls without creating alert storms.

Bulkhead: isolate capacity

Bulkhead prevents one slow dependency or workload from consuming all shared capacity. Isolation may use separate thread pools, semaphores, connection pools, queues, partitions, or service instances.

```
request capacity
  +-- inventory: 20 permits
  +-- payments:  10 permits
  +-- reports:    5 permits

reports saturate, but payments retain capacity
```

Thread-pool bulkheads isolate threads and queues but add scheduling and context-propagation costs. Semaphore bulkheads limit concurrency without moving work to another pool, which is often suitable for nonblocking or virtual-thread code. Virtual threads make blocked threads cheaper; they do not make downstream connections, database locks, or vendor quotas unlimited. Capacity limits still matter.

Reject promptly when a bulkhead is full or queue only within a bounded latency budget. An enormous queue converts overload into slow failure and consumes memory.

Transactional Outbox: make state change and event intent atomic

Writing a database row and publishing a message are two separate systems. If the service commits the row and crashes before publishing, downstream services never learn about the change. If it publishes first and the database rolls back, consumers see an event for state that does not exist.

The outbox pattern writes the domain change and an event record in the same local database transaction. A relay later publishes unsent records.

```
application transaction
  +-- update order
  +-- insert outbox event
          |
          v
relay reads event -> broker -> idempotent consumers
          |
          +-- mark published / advance durable position
```

The relay can still publish twice if it crashes after broker acknowledgement but before marking the record. Consumers therefore need idempotency. Plan retention, ordering, partition keys, schema evolution, relay lag metrics, poison-event handling, and cleanup. Change-data-capture can implement the relay, but it does not remove these contract decisions.

Composing resilience patterns

Pattern order changes resource usage and observations. A common logical flow is:

```
caller deadline
   -> bulkhead admission
      -> circuit breaker permission
         -> retry policy
            -> timeout per attempt
               -> remote adapter
```

This is a starting point, not a universal formula. If a breaker counts every retry attempt, it reacts differently than if it counts one logical operation. Holding a bulkhead permit across backoff preserves strict concurrency but may waste scarce capacity. Releasing it may allow more logical requests to create attempt bursts. Define what each metric and limit counts.

Resilience cannot be bolted on after the API is designed. You need idempotency keys, deadlines, cancellation propagation, bounded queues, and error classification in the contracts. Test with latency and partial failure, not only immediate exceptions.

How Spring expresses these patterns

Spring uses pattern ideas throughout its API. Dependency injection supplies strategies and handlers. `JdbcTemplate` and similar template classes preserve workflows while callbacks customize steps. Application events implement observer-like communication. Handler mappings and interceptors create mediated chains. Repositories and persistence contexts represent domain persistence patterns. Proxies apply transactions, caching, security, and aspects. Factories, adapters, and decorators connect implementations.

Recognizing these ideas is useful, but annotations do not remove their trade-offs. An asynchronous event changes transaction visibility. A transactional proxy has an invocation boundary. A repository method can still create N+1 queries. A retry annotation can repeat a non-idempotent side effect. Explain the runtime behavior behind the annotation.

Selecting technical options when AI assists development

AI can propose plausible Java patterns and libraries quickly, but correctness remains an engineering responsibility. Treat generated advice as a candidate, not an authority.

Start with the forces and constraints: Java and Spring versions, throughput, consistency, security, operational skill, deployment model, and failure budget. Check version-sensitive claims against official Java enhancement proposals, Javadocs, Spring reference documentation, and library release notes. Build the smallest executable proof for uncertain behavior, especially transactions, proxying, serialization, concurrency, and retry. Add tests that demonstrate the required invariant and failure behavior.

Record significant choices in a short architecture decision record: context, considered options, decision, consequences, and evidence. Review generated dependencies for maintenance, licensing, vulnerabilities, and fit with the existing platform. Use static analysis, code review, load tests, failure injection, and production telemetry to verify the option under realistic conditions.

In an interview, a strong answer is: "I use AI to broaden and accelerate exploration, then validate recommendations against primary documentation, a focused proof, automated tests, and production constraints. I remain accountable for the trade-off and record why the selected option is correct for this system." This demonstrates judgment without pretending either that AI is infallible or that experienced engineers never use it.

Common senior-level traps

- Naming an interface Strategy when there is no meaningful variation.
- Building deep Template Method inheritance with undocumented hooks.
- Treating queued commands as exactly-once execution.
- Leaving Chain continuation and ordering implicit.
- Modeling state objects without enforcing concurrent transitions in storage.
- Publishing synchronous events without defining listener failure behavior.
- Using an unbounded executor as an asynchronous event bus.
- Translating arbitrary Java Specifications into SQL by assumption.
- Creating a generic Repository that exposes persistence mechanics everywhere.
- Retrying non-idempotent operations or deterministic failures.
- Nesting retries and multiplying remote attempts.
- Using a circuit breaker as a substitute for timeouts or capacity limits.
- Assuming virtual threads remove the need for bulkheads.
- Treating the outbox as exactly-once delivery instead of atomic event intent.
- Accepting AI-generated pattern vocabulary without verifying runtime behavior.

A strong interview answer

I select behavioral patterns from the force that varies. Strategy replaces policy through composition; Template Method preserves a controlled workflow; Command makes intent explicit and versionable; Chain defines an ordered pipeline; State owns legal lifecycle behavior; and Observer communicates facts with explicit delivery semantics. Specification gives business rules names and composition, while Repository and Unit of Work protect aggregate and transaction boundaries. For remote dependencies, retries handle bounded transient failure, circuit breakers stop repeated calls, and bulkheads isolate capacity. The outbox atomically records state change and event intent, with duplicate-tolerant publication and consumption. I verify ordering, idempotency, concurrency, lifecycle, and observability rather than assuming the pattern name provides correctness.

Chapter summary

Behavioral, domain, and resilience patterns give a Java system explicit places for policy, workflow, state, communication, persistence, and failure control. Their power comes from making important decisions visible. Their danger comes from hiding operational semantics behind familiar names. Senior design connects the in-memory object model to transaction boundaries, concurrency, delivery guarantees, deadlines, capacity, and production evidence.

Revision Notes

- Strategy replaces an algorithm or policy through composition.
- Lambdas suit small stateless strategies; named classes suit substantial behavior.
- Template Method preserves a workflow skeleton through controlled inheritance.
- Prefer composition when workflow steps need independent runtime variation.
- Command represents intent as a value and needs versioning across boundaries.
- Queue delivery is commonly at least once; handlers need idempotency.
- Chain of Responsibility requires explicit order and continuation semantics.
- State centralizes lifecycle-dependent behavior and legal transitions.
- Storage locking or versioning must enforce concurrent transitions.
- Observer publishes facts; synchronous and asynchronous delivery have different failure models.
- Domain events are facts in past tense; commands request action.
- Mediator coordinates interaction but should not own every business rule.
- Iterator consistency may be fail-fast, snapshot, or weakly consistent.
- Specification names and composes rules; SQL translation needs an expression model.
- Repository speaks in aggregate and domain terms.
- JPA persistence context behaves like a Unit of Work and identity map.
- Retry only classified transient failures within an end-to-end deadline.
- Retry requires idempotency, backoff, jitter, and a single attempt budget.
- Circuit Breaker fails fast after repeated observed failure.
- Breaker states are Closed, Open, and Half-Open.
- Bulkhead isolates scarce capacity; virtual threads do not remove downstream limits.
- Transactional Outbox atomically records state change and event intent.
- Outbox relay and consumers must tolerate duplicates.
- Pattern composition order changes behavior and metrics.
- Validate AI suggestions with primary documentation, proofs, tests, and production constraints.

Revision Notes - Interview Answers

Likely interview question: Strategy versus Template Method?

Strategy uses composition to replace a complete policy and can vary at runtime. Template Method uses inheritance to preserve a fixed workflow while subclasses override selected steps. Prefer Strategy when algorithms vary independently; use Template Method when subclasses genuinely share a stable lifecycle with narrow hooks.

Likely interview question: How do Retry, Circuit Breaker, and Bulkhead differ?

Retry repeats a transiently failed operation when another attempt may succeed safely. Circuit Breaker stops calls temporarily after repeated failure. Bulkhead reserves separate capacity so one workload cannot exhaust the whole process. They solve different problems and often compose with timeouts, deadlines, and idempotency.

Likely interview question: Does Transactional Outbox provide exactly-once delivery?

No. It makes the database change and the intention to publish atomic in one local transaction. The relay can publish a record more than once, so messages need stable identifiers and consumers must deduplicate or perform idempotent state changes.

Likely interview question: Repository versus DAO?

A Repository models collection-like access to aggregates using domain language and protects aggregate boundaries. A DAO is usually closer to tables, records, or persistence operations. The terms are sometimes used loosely, so explain the interface, ownership, and transaction behavior rather than relying on the label.

Likely interview question: How do you validate a Java design suggested by AI?

I compare it with the system's constraints and supported Java/Spring versions, verify version-sensitive claims in primary documentation, create a focused proof for uncertain runtime behavior, encode invariants and failure cases in tests, review security and dependency risk, and record the evidence and trade-off in an architecture decision. AI accelerates exploration; the engineering team owns correctness.

Final Pattern Selection Checklist

Before proposing a pattern in an interview or design review, work through these checks:

- Name the concrete force that changes. Do not begin with the pattern name.
- State the invariant the design must protect under success, failure, and concurrency.
- Describe the simplest design without the pattern and the specific pressure it cannot handle cleanly.
- Draw the runtime call path, including proxies, decorators, queues, transactions, remote calls, and thread changes.
- Define ownership of state, connections, executors, subscriptions, cached data, and shutdown.
- Classify errors into retryable, permanent, rejected, and unknown-outcome categories.
- Establish idempotency, ordering, consistency, timeout, cancellation, and capacity semantics.
- Check whether the abstraction leaks framework, vendor, transport, or persistence types.
- Verify the pattern with a focused test for its main promise and a failure test for its main risk.
- Add telemetry that exposes logical operations, physical attempts, queueing, rejection, state changes, and lag.
- Record why the extra abstraction is worth its learning, debugging, and operational cost.
- Revisit the decision when measurements or platform capabilities change.

A mature answer does not claim that one pattern is always best. It establishes context, chooses a bounded responsibility, and explains consequences. If a constructor, enum switch, plain function, or ordinary composition already makes the rule clear, use it. The best pattern is often the one that leaves future readers with the fewest hidden assumptions.